

УНИВЕРСИТЕТ ИТМО
ФАКУЛЬТЕТ ПРОГРАММНОЙ ИНЖЕНЕРИИ И КОМПЬЮТЕРНОЙ ТЕХНИКИ

Лабораторная работа №6 по дисциплине «Вычислительная математика»

Вариант 13

Выполнил:

Снагин Станислав Максимович,
Р3215, 467525

Проверил:

Наумова Надежда Александровна

г. Санкт-Петербург, Россия
2026

Содержание

1	Задание	1
2	Выполнение работы	2
2.1	Цель работы	2
2.2	ОДУ	2
2.3	Рабочие формулы	2
2.3.1	Усовершенствованный метод Эйлера	2
2.3.2	Метод Милна	3
2.4	Описание алгоритма решения задачи	3
3	Заключение	13

1 Задание

1. Численные методы решения обыкновенных дифференциальных уравнений (ОДУ) должны быть реализованы в виде отдельного класса/метода/функции;
2. Пользователь выбирает ОДУ вида $y' = f(x, y)$ (не менее трех уравнений) из тех, которые предлагает программа;
3. Предусмотреть ввод исходных данных с клавиатуры: начальные условия $y_0 = y(x_0)$, интервал дифференцирования $[x_0, x_n]$, шаг h , точность ε ;
4. Для исследования использовать одношаговые методы и многошаговые методы (см. табл. 1);
5. Составить таблицу приближенных значений интеграла дифференциального уравнения, удовлетворяющего начальным условиям, для всех методов, реализуемых в программе;
6. Для оценки точности одношаговых методов использовать правило Рунге;
7. Для оценки точности многошаговых методов использовать точное решение задачи:

$$\varepsilon = \max_{0 \leq i \leq n} |y_i^{\text{точн}} - y_i|;$$

8. Построить графики точного решения и полученного приближенного решения (разными цветами);
9. Программа должна быть протестирована при различных наборах данных, в том числе и некорректных;
10. Проанализировать результаты работы программы.

2 Выполнение работы

2.1 Цель работы

Целью работы является решение задачи Коши для обыкновенных дифференциальных уравнений (ОДУ) первого порядка численными методами, исследование точности полученных результатов и сравнение численного решения с аналитическим.

2.2 ОДУ

Были придуманы следующие уравнения. Для каждого существует аналитическое решение.

$$1. \quad y' = 3y \quad y(x) = y_0 \cdot e^{3(x-x_0)}$$

$$2. \quad y' = y + 2x \quad y(x) = (y_0 + 2x_0 + 2) \cdot e^{x-x_0} - 2x - 2$$

$$3. \quad y' = 4xy \quad y(x) = y_0 \cdot e^{2(x^2-x_0^2)}$$

$$4. \quad y' = \frac{y}{x+1} \quad y(x) = y_0 \cdot \frac{x+1}{x_0+1}$$

2.3 Рабочие формулы

Метод Эйлера

Метод основан на замене производной конечной разностью. Формулы для вычисления:

$$y_{i+1} = y_i + h \cdot f(x_i, y_i)$$

где y_0 задано начальным условием, h — шаг, $f(x, y)$ — правая часть ОДУ.

2.3.1 Усовершенствованный метод Эйлера

Данный метод является неявным, для его реализации используется итерационный процесс (прогноз-коррекция):

1. Предсказание (предиктор):

$$\tilde{y}_{i+1} = y_i + h \cdot f(x_i, y_i)$$

2. Уточнение (корректор):

$$y_{i+1} = y_i + \frac{h}{2} [f(x_i, y_i) + f(x_{i+1}, \tilde{y}_{i+1})]$$

Итерации до $|y_{i+1} - \tilde{y}_{i+1}| \leq \varepsilon$

2.3.2 Метод Милна

Многошаговый метод прогноза-коррекции. Для начала работы метода необходимо иметь значения в четырех предыдущих точках.

1. **Этап прогноза:**

$$y_i^{\text{прогн}} = y_{i-4} + \frac{4h}{3}(2f_{i-3} - f_{i-2} + 2f_{i-1})$$

2. **Этап коррекции:**

$$y_i^{\text{корр}} = y_{i-2} + \frac{h}{3}(f_{i-2} + 4f_{i-1} + f(x_i, y_i^{\text{прогн}}))$$

где $f_k = f(x_k, y_k)$.

2.4 Описание алгоритма решения задачи

Алгоритм программы реализует численное решение задачи Коши $y' = f(x, y)$ и состоит из четырех этапов:

1. **Ввод и валидация:** принимаются начальные условия (x_0, y_0) , параметры интервала $[x_0, x_n]$, шаг h и точность ε . Проверяется корректность данных (шаг $h > 0$, интервал не нулевой) для предотвращения ошибок вычислений.
2. **Вычисления:**
 - *Одношаговые методы (Эйлер, усовершенствованный Эйлер):* выполняют цикл по шагам сетки, вычисляя y_{i+1} через y_i . Усовершенствованный метод дополнительно использует итерации (предиктор-корректор).
 - *Многошаговый метод (Милн):* сначала вычисляет y_1, y_2, y_3 с помощью метода Рунге-Кутты 4-го порядка. Затем запускает цикл, где значение прогнозируется и корректируется до выполнения условия точности ε .
3. **Оценка погрешности:** для методов Эйлера используется правило Рунге (сравнение решения с шагом h и $h/2$). Для метода Милна точность оценивается путем сравнения приближенного результата с точным аналитическим решением.
4. **Вывод результатов:** полученные массивы точек (x_i, y_i) передаются на клиентскую часть, где формируется итоговая таблица и строится график с помощью библиотеки Chart.js.

Программная часть

Исходный код доступен здесь – <https://github.com/SE-Itmo-Labs/nm-sem4-lab6>

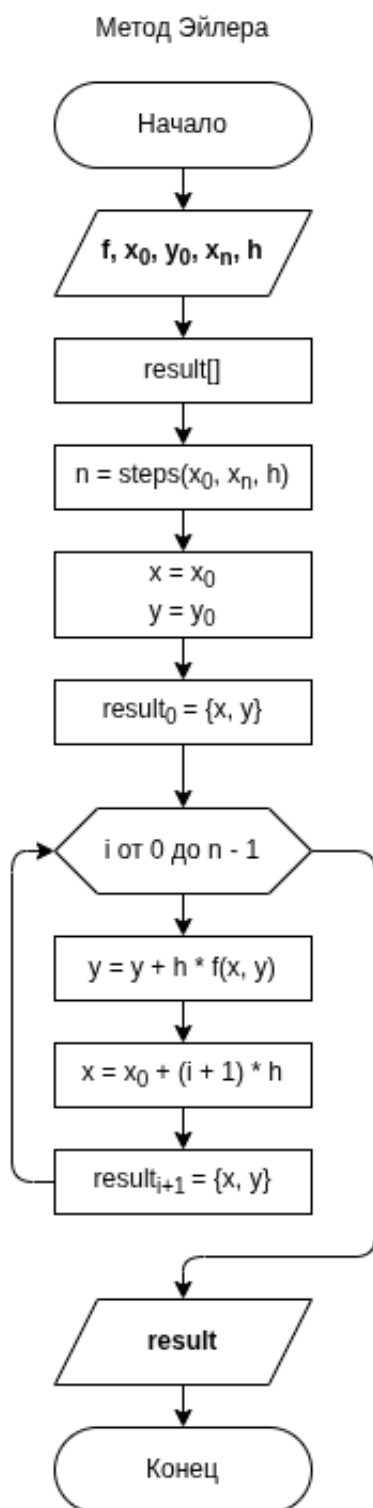


Рис. 1: Блок-схема метода Эйлера

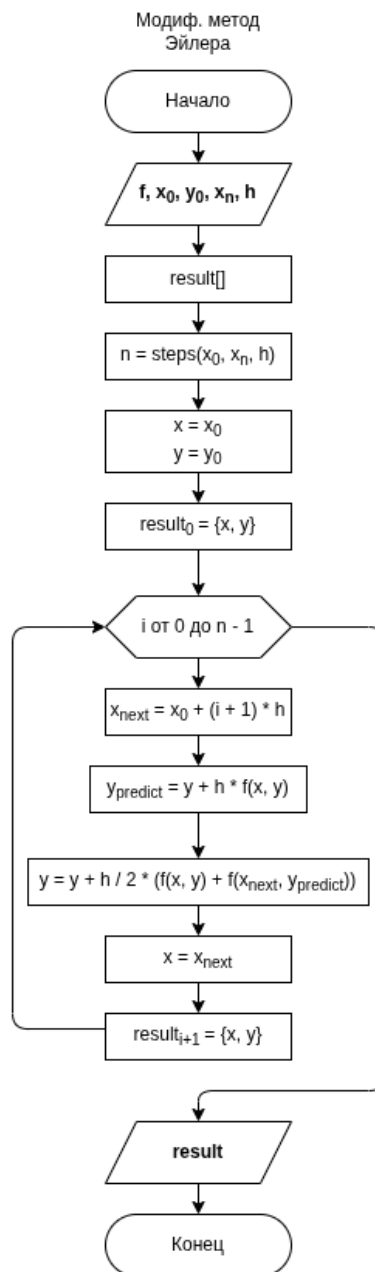


Рис. 2: Блок-схема модифицированного метода Эйлера

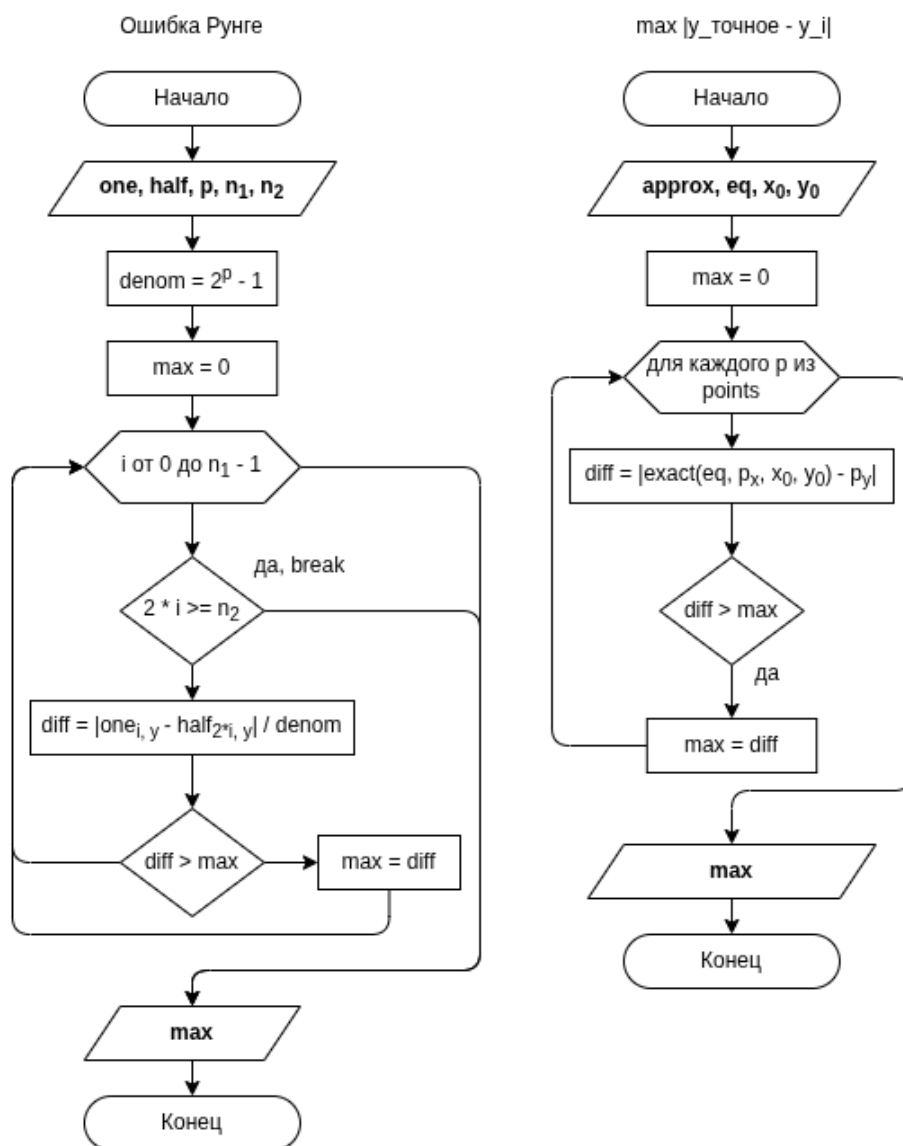


Рис. 3: Блок-схема с расчетами ошибок

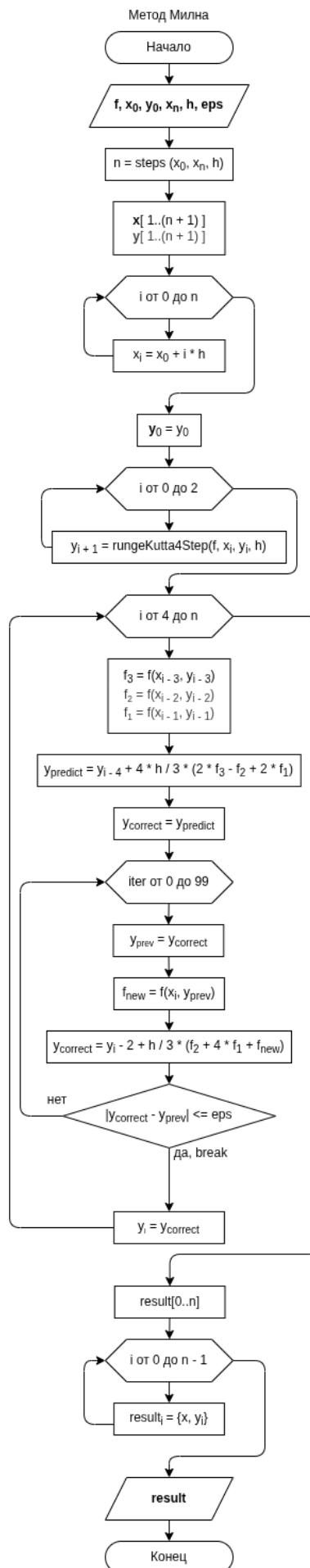


Рис. 4: Блок-схема метода Милна

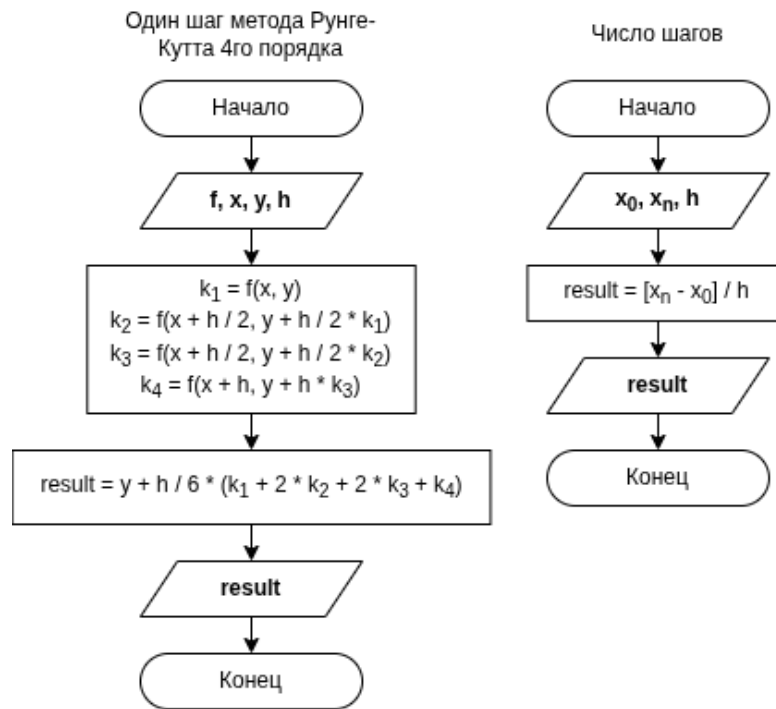


Рис. 5: Блок-схемы с прочими функциями

Листинг функций программы

```

1 /** МетодЭйлера */
2 public List<Point2D> euler(Rhs f, double x0, double y0, double xn, double h
3 ) {
4     List<Point2D> result = new ArrayList<>();
5     int n = steps(x0, xn, h);
6
7     double x = x0;
8     double y = y0;
9     result.add(new Point2D(x, y));
10
11     for (int i = 0; i < n; i++) {
12         y = y + h * f.f(x, y);
13         x = x0 + (i + 1) * h;
14         result.add(new Point2D(x, y));
15     }
16     return result;
17 }

```

```

1 /** УсовершенствованныйметодЭйлера */
2 public List<Point2D> improvedEuler(Rhs f, double x0, double y0, double xn,
3 double h) {
4     List<Point2D> result = new ArrayList<>();
5     int n = steps(x0, xn, h);
6
7     double x = x0;
8     double y = y0;
9     result.add(new Point2D(x, y));

```

```

10     for (int i = 0; i < n; i++) {
11
12         double xNext = x0 + (i + 1) * h;
13         double yPredict = y + h * f.f(x, y);
14
15         y = y + h / 2 * (f.f(x, y) + f.f(xNext, yPredict));
16         x = xNext;
17
18         result.add(new Point2D(x, y));
19     }
20     return result;
21 }

```

```

1  /** ОдиншагметодаРунгеКутты - го4- порядка */
2  public double rungeKutta4Step(Rhs f, double x, double y, double h) {
3      double k1 = f.f(x, y);
4      double k2 = f.f(x + h / 2, y + h / 2 * k1);
5      double k3 = f.f(x + h / 2, y + h / 2 * k2);
6      double k4 = f.f(x + h, y + h * k3);
7
8      return y + h / 6 * (k1 + 2 * k2 + 2 * k3 + k4);
9  }

```

```

1  public List<Point2D> milne(Rhs f, double x0, double y0, double xn, double h
2      , double eps) {
3
4      int n = steps(x0, xn, h);
5
6      double[] x = new double[n + 1];
7      double[] y = new double[n + 1];
8      for (int i = 0; i <= n; i++) x[i] = x0 + i * h;
9
10     y[0] = y0;
11     for (int i = 0; i < 3; i++) {
12         y[i + 1] = rungeKutta4Step(f, x[i], y[i], h);
13     }
14
15     for (int i = 4; i <= n; i++) {
16         double f3 = f.f(x[i - 3], y[i - 3]);
17         double f2 = f.f(x[i - 2], y[i - 2]);
18         double f1 = f.f(x[i - 1], y[i - 1]);
19
20         double yPredict = y[i - 4] + 4 * h / 3 * (2 * f3 - f2 + 2 * f1);
21
22         double yCorrect = yPredict;
23
24         for (int iter = 0; iter < 100; iter++) {
25             double yPrev = yCorrect;
26             double fNew = f.f(x[i], yPrev);
27             yCorrect = y[i - 2] + h / 3 * (f2 + 4 * f1 + fNew);
28
29             if (Math.abs(yCorrect - yPrev) <= eps) {
30                 break;
31             }
32         }
33     }
34 }

```

```

30     }
31
32     y[i] = yCorrect;
33 }
34
35 List<Point2D> result = new ArrayList<>();
36 for (int i = 0; i <= n; i++) result.add(new Point2D(x[i], y[i]));
37 return result;
38 }

```

```

1 public double rungeError(List<Point2D> one, List<Point2D> half, int p, int
  n1, int n2) {
2     double denom = Math.pow(2, p) - 1;
3     double max = 0.0;
4     for (int i = 0; i < one.size(); i++) {
5
6         if (2 * i >= half.size()) {
7             break;
8         }
9
10        double diff = Math.abs(one.get(i).y - half.get(2 * i).y) / denom;
11        if (diff > max) {
12            max = diff;
13        }
14    }
15    return max;
16 }

```

```

1 public double exactError(List<Point2D> approx, int eq, double x0, double y0
  ) {
2     double max = 0.0;
3
4     for (Point2D p : approx) {
5
6         double diff = Math.abs(exact(eq, p.x, x0, y0) - p.y);
7
8         if (diff > max) max = diff;
9     }
10    return max;
11 }

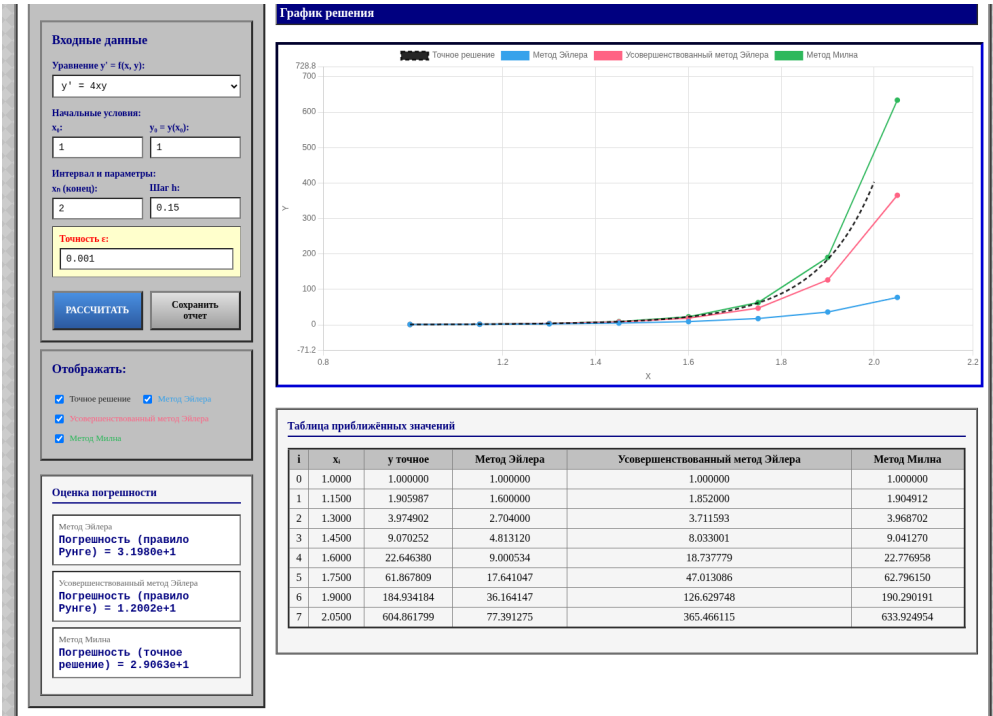
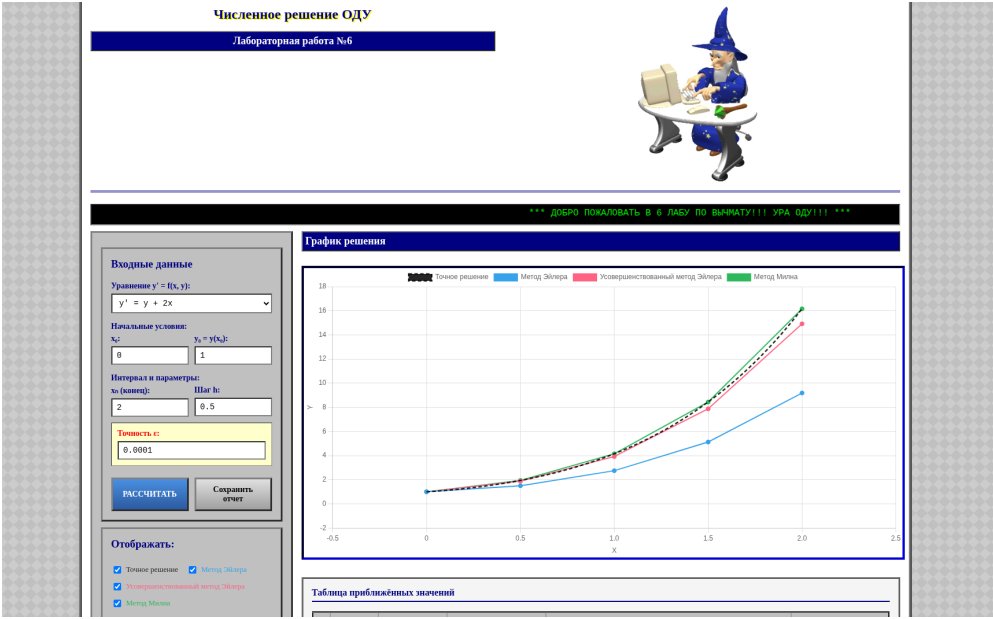
```

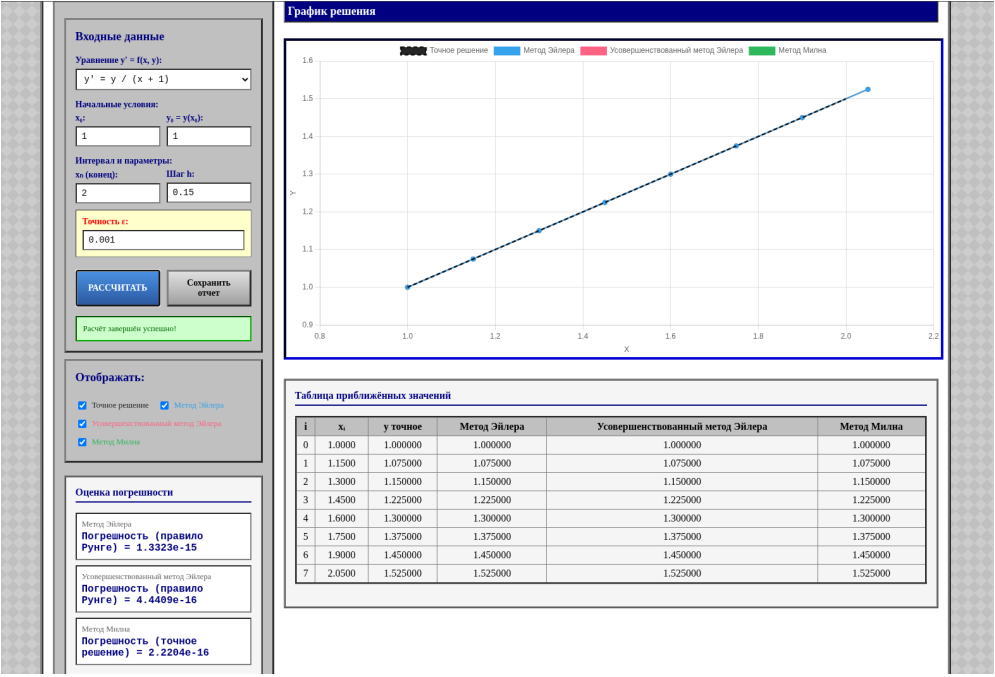
```

1 /** Числошаговсетки    n = (xn - x0) / h. */
2 public int steps(double x0, double xn, double h) {
3     return (int) Math.round((xn - x0) / h);
4 }

```

Примеры использования программы





3 Заключение

В ходе выполнения лабораторной работы были изучены и практически реализованы численные методы решения задачи Коши для ОДУ первого порядка (методы Эйлера (классический и усовершенствованный), Милна). Разработанное приложение позволило наглядно визуализировать графики решений, построить сравнительные таблицы и проанализировать влияние величины шага и параметров валидации на устойчивость и точность каждого из методов.